

LAUNCH26



Organized by
IEEE Computer Society Chapter
University of Kelaniya

The Relic Ring Protocol

Executive Summary

The Relic Ring Protocol challenges task participants with developing an efficient routing protocol to reconnect the Zeta-26 star system using primitive, fragmented legacy infrastructure. The system must account for physical propagation delays, disparate planetary data dialects, and maintain resilience in the face of hardware failure.

Background: The Silence of Zeta-26

For centuries, the Zeta-26 star system flourished under the Aether-Net, a quantum entanglement network that provided instant, zero-latency communication. Then came the Hyper-Flare of 3704. A graviton super-storm permanently fractured the quantum alignment of the network, leaving billions of people isolated on their respective worlds.

Hope now lies in the "Relic Ring"—a crude, physical infrastructure of underground fiber cables and laser transceivers built by the system's earliest pioneers. This legacy network survived the flare, but it is primitive, fragmented, and plagued by physical constraints.

Your Mission: Develop a ruthlessly efficient routing protocol to reconnect the star system. Your system must account for physical propagation delays, handle disparate planetary data dialects, and maintain resilience in the face of hardware failure.

1. Technical Objectives & Requirements

Your software must simulate a communication network based on a provided `universe-config.json` file. The universe is treated as a static entity in this task. The system must implement the following physical and logical mechanisms:

Universe-level physical constants (speed of light, tower delay, L_{max} , fiber speed fraction) are defined in `universe_metadata` and must be read from config, not hardcoded — the values in this document describe the defaults if not provided in the metadata.

A. Physical Propagation & Latency

The protocol must calculate latency based on four distinct components:

- **Subsurface Fiber Transit:** Data travels along the planet's equatorial fiber ring at $0.67c$.
- **Processing Tower Delay:** Every routing tower hit incurs a fixed processing penalty of $\Delta t = 7$ ms. (Even the receiving and sending towers if its the same tower that does both it is hit only once)
- **Atmospheric Refraction:** Signals piercing the ionized atmospheric shell (h) are slowed by the local refraction index (n).
- **Void Transmission:** Laser transmission across the vacuum (L) between planets.

Key Constraints:

- **Wireless Signal Threshold:** A single hop across the void cannot exceed $L_{\max} = 50,000,000$ km. If the distance is greater, the system must route through intermediate planets. If no planet bridges the gap, the route is reported as undeliverable.
- **Speed of Light (c):** 300,000 km/s.

B. Data Translation & Encoding

The network is a patchwork of incompatible dialects.

- **Codex Conversion:** Each planet uses a unique numerical base (codex) for receiving.
- **Transmission Flow:** Raw Payload $\rightarrow$ Next Hop Codex $\rightarrow$ Binary Stream $\rightarrow$ Void $\rightarrow$ Destination Codex $\rightarrow$ Local Decoding.
- **Internal Transit:** Within a planet, messages are converted to ASCII for routing between towers.

C. Node Schema Definition

The `universe-config.json` define each planet (node) using the following schema:

- **id:** Unique string identifier for the planet.
- **codex:** Integer defining the numerical base for data receiving.
- **x / y:** Coordinates of the planet in the universe grid.
- **radius_km:** The physical radius of the planet in kilometers.
- **active_towers:** Integer mapping the total number of routing towers available. (will be more than or equal to 4)
- **atmosphere_thickness_km:** The size of the atmospheric shell surrounding the planet.
- **refraction_index:** Local atmospheric density coefficient affecting signal speed.

D. Routing & Resilience

- **Shortest Path:** Implement an algorithm to find the lowest-latency route.
- **Dynamic Rerouting:** The system must detect node or link failures in real-time and instantly route packets around the "dead zone" without data loss.
- **User Interaction:** The system must allow users to visualize the full packet path taken and interact directly with nodes.

2. Deliverables**A. Technical Implementation**

- **GitHub Repository:** Containing the source code and a comprehensive README.md covering setup, running instructions, and a justification of all assumed constants.
- **Configuration:** The system must parse universe-config.json dynamically; no hardcoded planetary values are permitted.

B. Demonstration Video (10-15 Minutes)

Your video must visually prove the protocol's functionality through the following milestones:

- **M1: Universe Initialization:** Show the system ingesting the config file and spinning up the architecture.
- **M2: Multi-Hop Proof:** Trace a packet from an origin to a distant destination, showing the visible encoding translations at each hop.
- **M3: Latency Breakdown:** Display a detailed breakdown of latency per component (fiber, towers, atmosphere, void).
- **M4: Chaos Test:** Manually "kill" a node or link and demonstrate the system's ability to dynamically route the next message.

3. Evaluation Criteria

Category	Weight	Key Metrics
Baseline Delivery	Critical	Successful end-to-end delivery, correct codex translation, and complete hop logs.
Latency Accuracy	High	Precise calculation of fiber, tower, refraction, and void delays.
Resilience	High	Successful detection and rerouting around dead nodes without crashing.
Routing Efficiency	Medium	Correct enforcement of $L_{\max}$ and implementation of a shortest-path algorithm.
Code Quality	Medium	Readability, consistent naming, defensive error handling, and dynamic config parsing.
Documentation	Medium	Clarity of README and technical accuracy.

4. Participation Rules

- **Team Size:** 3 to 6 members, all from the same university.
- **Eligibility:** Each participant may join only one team.
- **Registration:** Teams are final once registered.
- **Communication:** A designated Team Leader must act as the sole point of contact with the Organizing Committee.

The Organizing Committee reserves the right to disqualify any team for non-compliance with the protocol. Furthermore, the selection of contestants rests solely with the panel, whose final verdict remains absolute.

5. Multi-Hop Transmission Example ("Hello world")

This example demonstrates how the message "Hello world" travels from Planet A to Planet C via Planet B under the specified dialect and encoding requirements.

Phase 1: Origin at Planet A (Base 8)

The packet starts inside Planet A's infrastructure.

- **Internal Representation:** The raw payload "Hello world" is represented as ASCII bytes inside Planet A's routing system: [72, 101, 108, 108, 111, 32, 119, 111, 114, 108, 100].
- **Next Hop Codex (Base 5):** Before hitting the void, Planet A converts the data into the dialect of the next destination, Planet B. For example, the first character 'H' (ASCII 72) becomes 242 in Base 5 ($72 = 2 \times 25 + 4 \times 5 + 2 \times 1$). The entire payload becomes: [242, 401, 413, 413, 421, 112, 434, 421, 424, 413, 400].
- **Void Transmission Stream:** This Base 5 sequence is serialized into a flat binary stream to be beamed via lasers across the vacuum.

Phase 2: Relay at Planet B (Base 5)

The packet arrives at one of Planet B's towers.

- **Void Arrival:** Planet B receives the raw binary stream and reads it out as Base 5 values.
- **Local Decoding & Internal Transit:** To route or process the packet between local towers, Planet B decodes the Base 5 back into standard ASCII (e.g., 242 in Base 5 $\rightarrow$ ASCII 72 $\rightarrow$ 'H'). The message briefly lives as "Hello world" in ASCII within Planet B's hardware while being transferred to the next tower for transmission.
- **Next Hop Codex (Base 14):** Planet B determines the packet must go to Planet C (Base 14) next. It converts the ASCII values into Base 14. The character 'H' (ASCII 72) in Base 14 becomes 52 ($72 = 5 \times 14 + 2 \times 1$). The entire payload becomes: [52, 73, 7A, 7A, 7D, 24, 87, 7D, 82, 7A, 72] (where A = 10).
- **Void Transmission Stream:** This Base 14 data is serialized back into a binary stream and beamed into space toward Planet C.

Phase 3: Final Destination at Planet C (Base 14)

The packet reaches its final destination.

- **Void Arrival:** Planet C captures the laser binary stream and processes it as Base 14 values.
- **Local Decoding:** Planet C runs its final local decoding step, mapping the Base 14 values back to ASCII to present to the end user (e.g., 52 in Base 14 → ASCII 72 → 'H').
- **Payload Delivered:** The text emerges perfectly intact on the destination terminal: "Hello world".

6. System Assumptions

- **Geometry:** Planets are modeled as 2D circles. Each planet has `active_towers` towers (from config), placed at equal angular intervals starting from the top (positive y-axis). Tower indices are assigned in increasing angle order (clockwise), starting with Tower 0 at the top.
- **Processing Delay:** A processing delay of 7ms is incurred for every tower a packet hits.
- **Propagation Speed:** The speed of light (C) is 300,000 km/s, while fiber propagation follows an arc path at $0.67 \times C$.
- **Line of Sight:** Space transmission requires a clear line of sight. *The tower pair (one on each planet) whose positions minimize the straight-line void distance between them is used for sending and receiving.*
- **Atmosphere:** The planet's atmosphere should be considered as a constant refraction from ground to height defined in the universe map.
- **Coordinate Scaling:** All `x/y` node coordinates are expressed in abstract units and must be multiplied by `coordinate_scale_unit_km` (from `universe_metadata`) to obtain actual kilometers for all distance calculations; `radius_km` is already given in kilometers and must not be scaled.
- **Void Distance Simplification:** For the purposes of calculating void distance (L), tower angular position is not factored into the geometry — L is computed from planet center-to-center distance minus each planet's (`radius + atmosphere_thickness`), as given in the formula. The closest-tower-pair rule (Line-of-Sight) is used only to determine which tower physically sends/receives the signal — for `hop_log` reporting and for internal fiber-arc routing (`T_p`) — and does not alter the L calculation itself.
- **Atmospheric Transit Distance Simplification:** Atmospheric transit distance is taken as exactly `h (atmosphere_thickness_km)` for each planet, regardless of the actual transmission angle through the shell — i.e., the signal is treated as passing straight through the atmosphere at thickness `h`, not along a slant path. This is consistent with the Void Distance Simplification above: tower angular position still determines which tower sends/receives and feeds internal fiber-arc routing (`T_p`), but does not alter the `h` or `L` values used in the `T_v` latency formula.

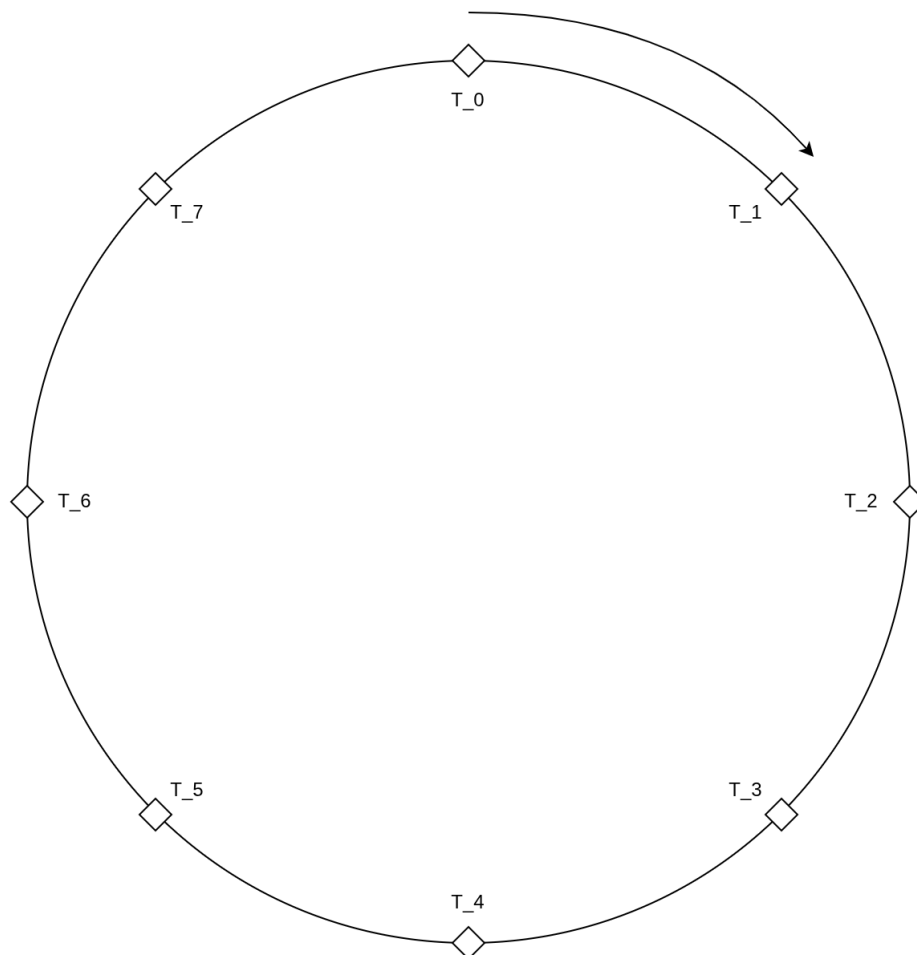
7. Packet Schema

The mandatory schema format that packets must have as they travel across the network grid:

- **origin_id**: Source planet node string.
- **destination_id**: Destination planet node string.
- **current_id**: Current planet node string.
- **payload**: The raw message content undergoing active conversion. (This should be translated into each dialect and shown)
- **hop_log**: An ordered array appended to by each relay node during transit to mathematically prove the route taken (per Tower info).

These are the mandatory fields — additional fields may be added as needed.

Tower Placement



Supporting Diagrams

